Selecting Reliable Mathematical Answers with Candidate Group Context

DASE7506 Advanced Machine Learning Methods | Mini Project 2 | 30 October 2026

Abstract

I investigate how to choose a correct numerical answer from four fixed language-model solutions under a small inference budget. The selected method combines supplied 1024-dimensional representations with text, numerical and bounded arithmetic features. A shared candidate encoder is followed by a scoring head that uses context from other candidates and from candidates with the same parsed answer. Training combines binary cross-entropy and correct-versus-incorrect pairwise ranking, with reduced weight for questions with no competing valid answer. The frozen model achieves **83.5118% on the official full test bank (390/467)**, 94.4862% on validation and 87.6056% on the difficult validation subset. Its complete inference bundle occupies 2,565,097 bytes (2.4463 MiB). Fresh-process prediction and independent scoring take 0.6941 seconds locally on an Apple M4 with four evaluation threads. Larger models and the tested OOF-based ensembles did not improve the selected validation result. The findings support candidate comparison and task-aligned weighting in this experiment, rather than capacity alone.

1 Research question and course motivation

I framed the project by connecting the classification-head approach in Lecture 4 with the distinction between convincing reasoning and verifiable answers in Lecture 5. Lecture 6 motivated answer agreement and preference ranking; Lecture 7 motivated separating correct choices from calibrated confidence [3]. The research question is whether a compact scorer benefits from judging a candidate in the context of its competitors, and whether more computation produces a useful accuracy gain. The specific architecture, features and experimental implementation were developed with Codex assistance, as disclosed in Section 8.

2 Task and data

The official protocol supplies a question, four candidate traces and aligned representations of shape [N, 4, 1024]. A predictor returns one Python integer in 0..3 for each question in bank order. Each question has at least one correct final answer; several candidates may be correct. The fixed parser and scorer compare final numerical answers, not every reasoning step [1, 2].

Table 1 Supplied non-overlapping splits

Split	Questions	Candidate solutions
Training	4,755	19,020
Validation	798	3,192
Test	467	1,868

Questions and references derive from GSM8K; candidate traces and representations were prepared with Qwen3-0.6B. The supplied license, notice and provenance manifest are retained. Selection accuracy is not a GSM8K answer-generation benchmark score.

3 Candidate features and context model

3.1 Reference-free feature construction

The final input has 2,228 features per candidate. Features depend only on the supplied question, candidate texts and representations. IDs are used only for alignment. The official parser normalizes final numbers before grouping, so equivalent fractions and decimals belong to the same answer group. Learned means and scales are fitted on training inputs only.

Table 2 Final candidate input

Feature block	Content	Size
Fixed representations	Raw vector and deviation from the mean of four vectors	2,048
Basic evidence	30 text, answer-frequency, numerical and similarity features, plus their within-question centered values	60
Extended evidence	60 arithmetic, number-relation and answer-group features, plus their within-question centered values	120
Total		2,228

Basic evidence includes text length, formatting counts, answer frequency, representation norm and cosine similarity to peers. Extended evidence includes equation residuals, numerical overlap between question and trace, simple numerical relationships to the parsed answer, and similarity within and across answer groups. Some features describe presentation as well as mathematical content; predictive value does not prove that a trace reasons correctly.

3.2 Bounded arithmetic evidence

A restricted arithmetic parser handles numeric literals, parentheses, basic operations and bounded powers, with limits on expression size, depth and intermediate values. It does not execute candidate code. Common mathematical notation is normalized; unsupported symbolic or ambiguous expressions are skipped. Unit conversions and formatting can otherwise create false violations, so checks are conservative. Their outputs are soft features rather than hard rejection rules: correct arithmetic can implement a wrong interpretation, and a partially incorrect trace can still contain the scorer-correct final number.

3.3 Shared encoding and candidate comparison

For candidate j of question i, standardized features are encoded into 256 dimensions. At inference, dropout is disabled:

$$\tilde{x}_{ij} = (x_{ij} - \mu) / \max(\sigma, 0.03), \quad h_{ij} = \mathrm{ReLU}(W_1 \tilde{x}_{ij} + b_1).$$

Let m_i be the mean hidden vector across four candidates, v_i their coordinate-wise maximum, and a_{ij} the mean hidden vector of candidates sharing j's parsed answer, including j. The head receives:

$$z_{ij} = [\, h_{ij} ; m_i ; v_i ; a_{ij} - h_{ij} \,], \quad z_{ij} \in R^{1024}.$$
$$s_{ij} = W_3 \, \mathrm{ReLU}(W_2 z_{ij} + b_2) + b_3.$$

The head has 64 hidden units and one output. It compares a candidate with its question group and answer group without positional encoding. There are 636,289 trainable parameters. Final selection uses the highest score among parseable candidates when any exist, with no additional answer-group score pooling. Permutation tests check that synchronously reordered candidates and embeddings preserve the selected numerical answer.

4 Learning and selection protocol

4.1 Multiple correct candidates

Labels are defined by the unchanged official numerical parser. A single correct index is not imposed when several candidates match the reference. Candidate correctness uses binary cross-entropy, with σ(s) denoting the sigmoid function:

$$L_i^{BCE} = -(1/4) \Sigma_j [y_{ij} \log \sigma(s_{ij}) + (1 - y_{ij}) \log(1 - \sigma(s_{ij}))].$$

For $P_i = \{(a,b) : y_{ia} = 1, y_{ib} = 0\}$, pairwise ranking encourages each correct candidate to score above each incorrect one. Correct candidates are not ranked against each other. The term is zero when P_i is empty:

$$L_i^{pair} = (1 / |P_i|) \Sigma_{(a,b) \in P_i} \log(1 + \exp(s_{ib} - s_{ia})).$$

4.2 Weighting genuinely competitive questions

Difficult denotes a question with at least two different valid parsed answers. Validation contains 355 such questions and 443 with one valid answer group. The test bank requires competing valid answers. Difficult-validation accuracy is therefore the primary development metric, with complete-validation accuracy breaking ties. The selected loss increases the relative emphasis on these competitive questions:

$$L = (1/N) \Sigma_i w_i(L_i^{BCE} + 0.5L_i^{pair}),$$
$$w_i = 1 \text{ for difficult questions, and } 0.25 \text{ otherwise.}$$

Table 3 Selected training configuration

Setting	Value
Optimizer and learning rate	AdamW; 0.0007
Weight decay and dropout	0.03; 0.25
Question batch size and gradient norm cap	128 questions; 5
Encoder and head widths	256 and 64
Random seed and selected epoch	17; epoch 10
Training and evaluation threads	2 for neural training; 4 for evaluation

4.3 Separation of fitting and evaluation

Training inputs and references fit all learned quantities. Validation selects configurations and checkpoints; neural searches check every five epochs. The selected run searched up to 70 epochs but retained epoch 10. OOF predictions use three question-level folds, with 3,170 training and 1,585 held-out questions per fold. All four candidates remain together, normalization is refitted on each training fold, and the selected epoch is fixed before OOF fitting. Nonnegative fusion weights and score scales use training OOF predictions; validation selects among resulting fusion settings. Test references are not used for training, tuning or method selection.

5 Experimental results

5.1 Comparisons from this development cycle

All comparisons below use experiments executed during this development cycle. Table 4 fixes seed 17 and encoder width 256; each configuration selects its checkpoint separately on validation. One changed decision is 0.1253 percentage points on complete validation and 0.2817 points on its difficult subset.

Table 4 Controlled configuration comparisons

Input and scoring design	Loss	Easy weight	Validation correct	Difficult correct
Combined features; independent	BCE	1	745/798	302/355
Extended features; independent	BCE	1	741/798	298/355
Extended features; independent	BCE + pair	1	740/798	297/355
Extended features; group context	BCE	1	749/798	306/355
Extended features; group context	BCE + pair	1	745/798	302/355
Extended features; group context	BCE + pair	0.25	**754/798**	**311/355**

Combined features comprise 2,108 representation and basic dimensions; extended features add 120 dimensions. Adding these features did not help the independent BCE model: it fell from 745 to 741 correct. Adding the pairwise term to that independent model also did not improve the result. The evidence therefore does not support a universal gain from arithmetic features or pairwise ranking alone.

Group context helped the compared extended-feature BCE configuration, reaching 749 correct. With context and the combined loss held fixed, reducing easy-question weight from 1 to 0.25 increased the difficult count from 302 to 311. This is consistent with emphasizing competing answers. However, separately selected epochs and limited validation size make this exploratory evidence, not an isolated causal estimate of one component.

5.2 Frozen result

Table 5 Selected model outcomes

Evaluation set	Correct / total	Accuracy
Complete validation	754 / 798	94.4862%
Difficult validation	311 / 355	87.6056%
Official complete test	390 / 467	**83.5118%**

After selection, code and inference assets were frozen and hashed before official full-test scoring. The test result was not used to revise features, weights or fusion. Other methods were not compared on test, so no claim is made of test-set superiority over them. The score evaluates final-answer selection, not the validity of every reasoning step.

Complete-validation accuracy is an optimistic proxy for the test task because 443 validation questions have no competing valid answer. The difficult subset is more relevant to that condition, but remains a development set and cannot replace the held-out test measurement.

5 Experimental results continued

5.3 Out-of-fold check and seed sensitivity

Question-grouped OOF predictions examine the selected recipe on all 4,755 training questions, with fewer questions per training fit. This is an additional check, not an untouched confirmation set: the architecture and epoch were already informed by validation.

Table 6 OOF performance of the selected recipe

OOF evaluation	Correct / total	Accuracy
All training questions	4,373 / 4,755	91.9664%
Difficult training questions	1,732 / 2,114	81.9300%

Across the three folds, difficult accuracy ranges from 80.8239% to 82.9787%. Full-training runs of the same 256-wide context-ranking recipe with seeds 17, 29 and 43 select 754, 748 and 750 validation answers correctly at pooling zero. This variation cautions against treating one selected seed as a precise estimate of general performance.

5.4 Capacity and alternative model families

Table 7 Alternatives selected using validation

Alternative	Validation correct	Validation accuracy
TF-IDF and structural linear model	729 / 798	91.3534%
Training PCA plus structural trees	737 / 798	92.3559%
Random Fourier feature model	731 / 798	91.6040%
Wide context model at 15.9461 MiB	746 / 798	93.4837%
Selected 2.4463 MiB model	**754 / 798**	**94.4862%**

The study ran 148 neural fits, 72 text/tree/PCA configurations and 18 random-feature configurations. These counts include repeated runs across widths, seeds, objectives and regularization settings. Neural alternatives included attention, larger widths and exponential moving averages. PCA, TF-IDF and all other fitted transforms used training data only.

Nonnegative fusion was fitted on OOF predictions rather than test labels. A representative five-member fusion achieved 82.6868% difficult OOF accuracy, above the selected single recipe's 81.9300%, but reached only 746/798 on validation (303/355 difficult). Tested fusion choices did not beat the selected single model. A training-OOF improvement alone was therefore insufficient to adopt a more expensive ensemble.

The 15.9461 MiB wide model was eligible by asset size but less accurate on validation. I retained the smaller model: the resource cap was a constraint, not a target to fill with parameters that failed to improve the selection criterion.

6 Reliability and failure analysis

Table 8 Validation by number of different valid answers

Different answers	Questions	Correct	Accuracy
1	443	443	100.0000%
2	227	208	91.6300%
3	108	88	81.4815%
4	20	15	75.0000%

Accuracy is lower in groups with more distinct answers in this validation bank. Agreement is useful context, but not proof of correctness. This is a descriptive association: group composition and question difficulty can differ, and the four-answer group has only 20 questions. The table does not establish that adding diversity itself causes errors.

6.1 Correct arithmetic with the wrong interpretation

One validation question combines three classes for opposite-gender dance partners. The selected candidate correctly totals 46 male students, 48 female students and 94 students overall. It then calculates $94 - 46 = 48$ as the number unable to partner. The arithmetic is valid, but 46 counts pairs, not paired students. The correct unpaired count is $|48 - 46| = 2$. Local equation checks cannot resolve this without understanding the quantities being counted.

6.2 An explicit arithmetic error still selected

Another validation question gives five daily counts of 3, 5, 7, 9 and 11. The selected trace sums them as 36, although the sum is 35. Arithmetic features did not guarantee rejection: other learned signals can dominate, and not every notation pattern is equally covered. A soft feature supplies evidence, not guaranteed verification.

6.3 Limits of the evidence

Scoring checks the final number, not the truth of every intermediate step. Fixed representations and local numeric features do not fully verify natural-language conditions or units. At least one validation question also permits potentially different wording/reference interpretations. Such observations did not change the supplied references or scoring rules.

Many configurations were selected using only 355 difficult validation questions, creating a risk of optimistic selection. Separately selected epochs prevent some comparisons from isolating a single factor. The official test still contains 77 incorrect selections; generalization to an instructor audit bank is unmeasured. Deterministic, reproducible implementation does not remove these limits.

Ranking scores have not been validated as correctness probabilities. No Brier score, reliability curve or risk-coverage evaluation is reported. High scores should not be presented as calibrated probabilities. The method is a learned selector, not a complete reasoning verifier.

7 Resource use and reproduction

Table 9 Frozen assets and local evaluation resources

Measurement	Recorded value
Weights plus configuration	2,565,097 bytes = 2.4463 MiB
Asset cap	16,777,216 bytes = 16 MiB
Fresh prediction process	0.6281 s
Fresh independent scorer	0.0661 s
Sum of the two process times	0.6941 s
Local hardware and thread limit	Apple M4; 16 GB RAM; 4 evaluation threads

Assets include weights, learned normalization statistics and configuration. Each file is counted conservatively using the larger of disk and decompressed size. Inference uses NumPy and threadpoolctl, without retraining, external model downloads or network access. Timing includes startup, imports, loading, feature extraction, prediction and independent scoring. Eligibility under the official 30-second CPU cap is determined on the reference Xeon Platinum 8457C, not this local machine.

The 148 neural fit-time records sum to 2,891.43 seconds. This aggregates overlapping fits and is not elapsed wall time; it excludes other families, OOF and preprocessing. Per-run settings and costs are retained. Reproducing only the selected ten-epoch fit took approximately 1.96 seconds locally with two threads, excluding the broader search.

7.1 Reproduce from the project root

Use Python 3.12. Start at the project root and run these commands. For checkpoint-only evaluation, install only inference requirements.

```
python3.12 -m venv code/.venv
cd code
.venv/bin/python -m pip install -r requirements-inference.txt
bash run.sh check_data.py
bash run.sh verify.py measure --checkpoint runs/final \
  --freeze runs/final-freeze.json --split test \
  --output runs/reproduction-test
```

For fixed-configuration retraining, install training requirements. The selected epoch and configuration are fixed; this does not rerun model selection.

```
.venv/bin/python -m pip install -r requirements-train.txt
MP2_TRAIN_THREADS=2 bash run.sh retrain_selected.py \
  --manifest runs/selected-training.json --output runs/retrained
```

Pinned versions: NumPy 2.5.3, threadpoolctl 3.7.0, PyTorch 2.15.0, SciPy 1.18.1 and scikit-learn 1.9.1. The official score is in `runs/reproduction-test/test-score.json`; asset accounting is in `runs/final-assets.json`. Full commands and exploration records are documented in the project README.

7 Resource use and reproduction continued

7.2 Version identity and implementation checks

The code version is identified by source hashes in `code/runs/final-freeze.json`. The checkpoint matches that frozen version. Its weight SHA-256 is:

```
e4d3530acb65f9e8b10efa74361740b9
2a9191682053f8e310b636f9bb36a3c0
```

The second line continues the first; the full digest is in the asset manifest. Fixed-epoch retraining reproduced all weight arrays and the checkpoint hash exactly on this machine, with identical validation scores and choices. This local reproduction does not guarantee bitwise equality across every platform.

The final suite passed 36 checks covering input sizes, candidate and question reordering, deterministic and non-mutating predictions, reference-file isolation, asset containment, decompressed-size accounting, arithmetic, and training-to-inference parity. Frozen code and assets are checked again after report preparation without modifying them.

8 Conclusion and contribution statement

A compact context-aware scorer achieved 83.5118% official full-test accuracy with 2.4463 MiB of assets. Among tested settings, context-aware scoring with reduced easy-question weight produced a better validation result than the independent scorers. Larger capacity and tested ensembles did not improve it. Semantic interpretation, reliable use of arithmetic evidence and confidence calibration remain unresolved; they are not solved merely by increasing model size.

Personal contribution and AI assistance

I formed the research question, overall direction and evaluation goals by connecting ideas from Lectures 4-7. Within that framework, I used OpenAI Codex to help extend the research direction into a concrete candidate-group model, feature schemes, training objectives and experimental procedures. Codex assistance covered technical exploration, code writing, training and evaluation, portable inference export, tests, result organization and report drafting. My contribution concerns the course-informed framing, research direction, review of experimental interpretation and final choices; I remain responsible for understanding and checking the submitted work.

Supplied data and official evaluation tools retain their attribution and licenses. Scientific Agent Skills supported evidence appraisal and reporting [4]; it is not a source of benchmark performance claims. Reported tables use results from this development cycle and the frozen official evaluation.

References

[1] DASE7506 MP2. *Task, Requirements and Assessment.* Fall 2026. github.com/xudongwu-0/DASE7506-MP2/blob/main/guide/GUIDE.md.

[2] DASE7506 MP2. *Code and Running Instructions.* Protocol dase7506-mp2-open-hard-verifier-v5. github.com/xudongwu-0/DASE7506-MP2/blob/main/code/README.md. Data attribution and revisions are in the supplied NOTICE and manifest.

[3] Chen, J. (2026). *DASE7506 Advanced Machine Learning Methods, Lectures 4-7.* Course materials supplied with the project workspace.

[4] Kassis, T., Agarwal, V., He, Y., Patel, D., and Brueckner, A. M. (2026). *Scientific Agent Skills: A Library of Procedural Knowledge for Research Agents.* arXiv:2609.00065. doi.org/10.48550/arXiv.2609.00065.